

RegExp & String Methods

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>re.test(str)</code>	Returns true if pattern matches in string, else false EXAMPLE: <code>/cat/.test('catnip')</code> → <code>true</code>
<code>re.exec(str)</code>	Returns an array of match info, or null. Updates <code>lastIndex</code> if <code>/g</code> EXAMPLE: <code>/\d+/.exec('id: 42')</code> → <code>['42']</code>
<code>str.match(re)</code>	Returns match array or null. Returns all matches if <code>/g</code> is set EXAMPLE: <code>'1 2 3'.match(/\d+/g)</code> → <code>['1', '2', '3']</code>
<code>str.matchAll(re)</code>	Returns an iterator of all matches including capture groups (requires <code>/g</code>) EXAMPLE: <code>[...str.matchAll(/\d+/g)]</code>
<code>str.search(re)</code>	Returns index of first match, or -1 if not found EXAMPLE: <code>'foo bar'.search(/bar/)</code> → <code>4</code>
<code>str.replace(re, repl)</code>	Replaces match(es) with replacement string or function EXAMPLE: <code>'a b c'.replace(/\w+/g, 'x')</code> → <code>'x x x'</code>
<code>str.replaceAll(re, repl)</code>	Replaces all matches. Regex must have <code>/g</code> flag EXAMPLE: <code>str.replaceAll(/\w+/g, 'x')</code>
<code>str.split(re)</code>	Splits string by matches into an array EXAMPLE: <code>'a, b; c'.split(/[;,]\s*/)</code> → <code>['a', 'b', 'c']</code>

JavaScript Regex Flags

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>g</code>	Global: Find all matches rather than stopping after the first EXAMPLE: <code>/a/g</code>
<code>i</code>	Ignore case: Case-insensitive matching EXAMPLE: <code>/a/i</code>
<code>m</code>	Multiline: <code>^</code> and <code>\$</code> match start/end of line instead of whole string EXAMPLE: <code>/^foo/m</code>
<code>s</code>	DotAll: <code>.</code> matches newlines as well EXAMPLE: <code>/foo.bar/s</code>
<code>u</code>	Unicode: Treat pattern as a sequence of Unicode code points EXAMPLE: <code>/\u{1F600}/u</code>
<code>v</code>	Unicode sets: Enables unicode property escapes and set operations EXAMPLE: <code>/[\p{L}&&[^a-z]]/v</code>
<code>y</code>	Sticky: Matches only from the index indicated by the <code>lastIndex</code> property EXAMPLE: <code>/foo/y</code>
<code>d</code>	Indices: Generate indices for substring matches (<code>match.indices</code>) EXAMPLE: <code>/foo/d</code>

JavaScript-Specific Syntax

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>/pattern/flags</code>	Regex literal syntax EXAMPLE: <code>const re = /ab+c/i;</code>
<code>new RegExp('pat', 'f')</code>	Constructor syntax (requires escaping backslashes) EXAMPLE: <code>new RegExp('\\d+', 'g')</code>
<code>(?<name>...)</code>	Named capturing group. Accessed via <code>match.groups.name</code> EXAMPLE: <code>/(?<year>\d{4})/.exec(s).groups.year</code>
<code>\k<name></code>	Backreference to a named group in the pattern EXAMPLE: <code>/(?<quote>["']).*?\k<quote>/</code>
<code>\$<name></code>	Named capture replacement in <code>str.replace()</code> EXAMPLE: <code>str.replace(/(?<v>\d+)/, '\$<v>px')</code>
<code>\$&, \$1, \$2</code>	Special replacement patterns: <code>\$&</code> is whole match, <code>\$n</code> is capture group EXAMPLE: <code>str.replace(/(\w+)/, '[\$1]')</code>
<code>\p{...}</code>	Unicode property escape (requires <code>/u</code> or <code>/v</code> flag) EXAMPLE: <code>/\p{Script=Greek}/u</code>